axiateam
RDT company

Módulo 7

Funciones y Tipado de Funciones

Curso: TypeScript

90 min

Objetivos de Aprendizaje

- Declarar **tipos para parámetros** y retornos de funciones
- Implementar funciones con **parámetros opcionales** y por defecto
- Utilizar **tipos de función** (function types)
- Trabajar con **callbacks** tipados
- Aplicar **overloads** de funciones

1. Contexto

Funciones en TypeScript

El Problema con JavaScript

```
// Sin tipos, sin documentación
function processUser(data) {
  return {
    id: data.id,
    name: data.name.toUpperCase(),
    active: data.active || true
  };
}
```

¿Qué pasa si `data.name` es undefined? ¿Qué tipo retorna la función?

La Solución con TypeScript

```
interface UserInput {
  id: string;
  name: string;
  active?: boolean;
}

interface UserOutput {
  id: string;
  name: string;
  active: boolean;
}

function processUser(data: UserInput): UserOutput {
  return {
    id: data.id,
    name: data.name.toUpperCase(),
    active: data.active ?? true
  };
}
```

2. Tipos de Parámetros

Parámetros Tipados

```
// Parámetros con tipos explícitos
function greet(name: string): string {
  return `Hola, ${name}!`;
}

// Múltiples parámetros
function createUser(name: string, age: number, active: boolean) {
  return { name, age, active };
}

// Parámetros opcionales (con ?)
function findUser(id: string, includeProfile?: boolean) {
  // includeProfile puede ser undefined
}

// Parámetros con valores por defecto
function paginate(items: string[], page: number = 1, limit: number = 10) {
  const start = (page - 1) * limit;
  return items.slice(start, start + limit);
```

Lab: Funciones del Sistema de Tareas

Objetivo: Implementar funciones CRUD con tipos completos.

1. Implementa `createTask` con tipos
2. Implementa `updateTaskStatus`
3. Implementa `getTasksByStatus`

Código del Lab

```
type TaskStatus = "pending" | "inProgress" | "completed";
type TaskPriority = "low" | "medium" | "high";

interface Task {
  id: string;
  title: string;
  status: TaskStatus;
  priority: TaskPriority;
  dueDate?: Date;
}

function createTask(
  title: string,
  priority: TaskPriority = "medium"
): Task {
  return {
    id: crypto.randomUUID(),
    title,
    status: "pending"
```

3. Tipos de Función

Function Types

```
// Tipo de función
type MathOperation = (a: number, b: number) => number;

const add: MathOperation = (a, b) => a + b;
const multiply: MathOperation = (a, b) => a * b;

// Callbacks tipados
function processItems<t>(
  items: T[],
  transform: (item: T) => T
): T[] {
  return items.map(transform);
}

const result = processItems([1, 2, 3], (x) => x * 2);</t>
```

Lab: Sistema de Procesamiento de Tareas

Objetivo: Crear funciones de orden superior tipadas.

1. Crea tipo para filtro de tareas
2. Crea tipo para transformador
3. Implementa función que acepta filtros

Resumen

Puntos Clave

- Los parámetros pueden ser **obligatorios, opcionales** o tener **valores por defecto**
- Los **rest parameters** usan el operador `...`
- Los **function types** definen la firma de una función
- Los **callbacks** pueden tiparse con genéricos
- **Overloads** permiten múltiples firmas

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Implementar funciones de negocio con tipos.

1. Crea función para calcular fecha de vencimiento de préstamo
2. Implementa función de filtrado con callbacks
3. Crea función con overloads para buscar libros

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- Declara tipos explícitos para parámetros y retornos siempre
- `?` para opcionales, `=` para valores por defecto
- Function overloads: firma más específica primero

⚠️ Gotchas

- Arrow functions capturan `this`
- Parámetros por defecto pueden ser `undefined`
- `never` retornan implícitamente si lanzas errores

Referencias

Recursos Adicionales

- Functions
- Function Overloads

Gracias!

Módulo 7 Completado

Siguiente: Módulo 8 - Clases, Interfaces y Herencia